

Measurement-Driven Optimization of Transformer Inference on Consumer GPU Hardware

TikTok TechJam 2026 — Problem 3: Implement a GPU Kernel for a Transformer Layer

Team Blackroom

Satsatranurak Phattarachai
Suthamkasem Thanwisit
Junchan Koompa
Tongdon-ngao Plengpin
Rojanawisoot Katunyoo

Nanyang Technological University

Target hardware	NVIDIA GeForce RTX 4080 Laptop GPU (Ada, sm_89)
Headline result	10.014x geometric-mean speedup, 13/13 shapes correct
Repository	github.com/koompaj/Tiktok_Techjam2026_Blackroom

August 2026

Abstract

We present an optimized inference path for a fixed Transformer-layer formula, evaluated against a reference PyTorch implementation across the fourteen shapes of the TechJam 2026 Problem 3 suite. Across the thirteen shapes for which a reference can be executed, the implementation achieves a geometric-mean speedup of 10.014x, ranging from 2.707x to 35.179x, while satisfying the task's per-element correctness contract on every shape. The fourteenth shape admits no runnable reference — its baseline would require a 20 TB attention-score tensor — and is demonstrated to execute correctly in isolation.

The central design decision is methodological. The problem statement invites per-shape implementations selected by shape checks; hardcoded thresholds encode one machine's characteristics as constants and mispredict elsewhere. Instead every per-shape choice — precision, execution path, slice size, fusion — is decided by timing the real candidates on the real input during the benchmark's untimed warmup. The resulting policy is not uniform: ten of thirteen shapes adopt CUDA-graph capture, three run eagerly; eleven adopt mixed fp16, two measure fp32 as faster. No constant in the source predicts that partition. We also report an ablation isolating operator fusion (1.31x across twelve shapes, winning on all twelve) and four defects that only measurement exposed.

1. Introduction

The Transformer architecture underpins modern language, vision, speech, and recommendation systems, and its inference cost is a direct operational expense at deployment scale. The dominant computational patterns — dense matrix multiplication, attention-score computation, softmax normalization, layer normalization, and feed-forward projection — stress different hardware resources, and which resource binds depends strongly on the input shape.

This heterogeneity is the crux. Batch size 1 at width 128 performs roughly 0.13 GFLOP across some sixty kernel launches - the GPU idles, waiting on the host. Batch size 10,000 at the same width is bandwidth-bound. $d_{\text{model}} = 1024$ is GEMM-bound. An optimization decisive for one is irrelevant, or harmful, for another.

1.1 Contributions

- **Performance.** An optimized forward path achieving 10.014x geometric-mean speedup across 13 shapes, correct on all of them under the task's tolerance.
- **Adaptivity.** A runtime calibration mechanism that selects precision, execution path, and fusion by direct measurement on the real input, rather than by hardcoded shape thresholds.
- **Feasibility.** Execution of a shape whose reference implementation is physically impossible to run, via memory-aware slicing and output aliasing.
- **Ablation.** A per-shape ablation isolating the fusion contribution, plus a five-mode study that overturned our own initial adoption policy.
- **Negative results.** Four defects found by measurement that code review did not surface, including two cases where our own first fix was wrong and was caught by re-measuring.

2. Background

For an input sequence X in $\mathbb{R}^{(N \times d)}$, the layer projects to queries, keys and values and computes scaled dot-product attention:

$$Q = X W_Q \quad K = X W_K \quad V = X W_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

The scaling keeps the dot products out of the softmax's saturating region. The structurally critical property for optimization is that $Q K^T$ has shape $[B, H, S, S]$, quadratic in sequence length: materializing it is what makes naive attention expensive in memory rather than merely in arithmetic. The reference is a faithful expression of this mathematics, and its cost follows from that faithfulness — it materializes every intermediate the formula names. Section 3 examines what that costs.

3. Problem Analysis: where the reference loses time

Optimization without a defect list is guesswork. Before writing any kernel we read the reference line by line and enumerated what it spends time on that the mathematics does not require. The mapping below is deliberately one-to-one, so every optimization is traceable to an observed defect rather than adopted because it is a known technique.

3.1 Anatomy of the reference forward pass

The reference attention computes, per layer:

```
scores = matmul(q, k.transpose(-2,-1)) * scale # [B,H,S,S] allocated
scores = scores.masked_fill(causal_mask, -inf) # half of it discarded
probs = softmax(scores.float(), -1).to(dtype) # [B,H,S,S] AGAIN, fp32
context = matmul(probs, v)
```

Two full [B, H, S, S] tensors are live simultaneously — the scores in the model dtype and the fp32 copy the stable softmax requires. That single line is the dominant memory cost of the whole layer, and it is quadratic in sequence length:

Shape	[B,H,S,S] elements	Both copies, fp32	Consequence
#1 (S=128)	4.2 M	34 MB	fits, but wasteful
#13 (S=1024)	268 M	2.1 GB	dominates the layer
#14 (S=100000)	5.1 T	~20 TB	cannot be allocated at all

Table 1: The cost of materializing the attention score matrix. Shape #14 is not slow in the reference; it is impossible.

The block adds a quieter cost. Each sublayer boundary is a separate full-tensor pass over the [B, S, d] activation — a LayerNorm, a residual add, and in mixed precision a dtype cast performing no arithmetic at all. Two boundaries per layer across four layers is up to twenty-four full passes before a single GEMM is counted.

3.2 Defect-to-solution map

The following is the complete list of what we found and what each led to. Section 4.2 describes the countermeasures; Section 7 quantifies the contribution of the fusions specifically.

#	Defect in the reference	Why it costs	Countermeasure
D1	[B,H,S,S] score tensor materialized, then again in fp32 for the softmax	Quadratic memory; at S=100000 it exceeds any GPU	Fused SDPA - never forms the matrix
D2	Full score matrix computed, then half overwritten with -inf	~50% of attention arithmetic done and discarded	is_causal=True - masked half never computed
D3	Padding mask applied where it cannot change the result	A mask tensor demotes SDPA and blocks graph capture	Prefix-mask elision, after proving it a no-op
D4	Q, K, V are three separate [d,d] GEMMs	Three launches, three suboptimal shapes	Fused QKV - one [3d,d] GEMM
D5	Every sublayer boundary is a full pass; the dtype cast does no arithmetic	Bandwidth cost scaling with depth, not useful FLOPs	Triton residual+LayerNorm+cast kernel
D6	FFN intermediate written, re-read for GELU, written again	Three passes over [B,S,ffn] for one activation	cuBLASLt GEMM+GELU epilogue - written once
D7	Small shapes issue ~60 launches for ~0.13 GFLOP	GPU idles on host submission; arithmetic is not the bottleneck	CUDA-graph capture - one replay
D8	Each layer re-reads the whole activation from HBM	At B=10000 nothing stays resident between layers	Slice-through-the-stack - stays L2-resident
D9	Everything runs in fp32	Tensor cores underused; flash backend refuses fp32	Measured per-shape mixed precision

Table 2: Every defect identified in the reference, and the countermeasure adopted for it. Nine defects, nine countermeasures.

Three things follow. Only D9 is a precision trade; the other eight are pure waste removal costing no accuracy. D1 alone changes what is possible rather than what is fast — it is the difference between shape #14 running and not. And D5-D8 are bandwidth or launch-overhead defects, which is why the largest speedups appear on the smallest shapes: there, arithmetic was never the bottleneck.

4. Methodology

4.1 Measure, do not hardcode

The problem statement permits shape checks. We avoided them: a threshold like "use fp16 when batch exceeds 32" is a constant fitted to one GPU's cache hierarchy, clocks and driver - unfalsifiable in the source and silently wrong elsewhere.

Instead, on first encountering a shape, the implementation builds the candidate configurations, runs them on the real input, and times them, caching the winner against the shape and weight fingerprint. This happens inside the benchmark's untimed warmup, so it does not contaminate reported latency. Section 8.1 shows the non-obvious partition it produces.

4.2 Optimizations applied

The nine countermeasures are named in the defect map above (Table 2). Two are worth expanding, because they are correctness arguments rather than engineering choices.

4.2.1 Prefix-mask elision

This is a correctness argument rather than an engineering trick. SDPA reaches its fastest backends only with `is_causal=True` and no explicit `attn_mask`; any mask tensor demotes it and makes the forward uncapturable. Under causal attention with prefix padding — valid tokens first — a padded key is only ever attended to by a later query, which is itself padded, and padded output rows are zeroed at the end of the block regardless. The mask is therefore a no-op and is dropped, after the prefix property is verified once per mask tensor.

4.2.2 Slice-through-the-stack

For large batches the activation exceeds cache capacity. Tiling within each sublayer bounds peak memory but re-reads the whole activation from HBM at every layer boundary. We instead slice over the batch and drive each slice through the entire stack before starting the next, so it stays resident from its first LayerNorm to its last. The budget comes from the device's reported L2 capacity rather than a constant. On shape #6 this is worth $\sim 3.3\times$ over per-sublayer tiling.

4.3 Precision policy

The correctness contract is a disjunction: an element passes if its absolute error is within 0.002 OR its relative error is within 2%. Because the output of the final LayerNorm has unit variance, a large fraction of elements sit near zero, where 2% of nearly nothing is nearly nothing. The absolute floor is therefore what binds, and the policy must be chosen against it.

Two facts justify reduced precision. The harness runs the reference itself with TF32 matmuls enabled by default, so the baseline already accumulates $\sim 5e-4$ relative rounding per GEMM; fp16 carries an 11-bit mantissa and cuBLAS accumulates in fp32, making the move lateral rather than a step down. And SDPA's flash backend refuses fp32 outright.

Configuration	Worst-case absolute error	Verdict
Our fp32 path	$0.67e-3 - 1.29e-3$	PASS
fp16 GEMMs + fp32 residual	$0.90e-3 - 1.88e-3$	PASS — adopted
fp16 everywhere	$6e-3 - 8e-3$	FAIL: near-zero elements miss both branches
bf16 GEMMs + fp32 residual	$1.0e-2$	FAIL

Table 3: Measured worst-case absolute error against the fp32 reference across the 13 comparable shapes.

Carrying the residual in fp16 too is ~5x worse: each layer's rounding then compounds into the next instead of being absorbed by an fp32 accumulator. bf16 fails despite its exponent range — eight mantissa bits are not enough. The fp16 margin is thin: 1.88e-3 against a 2.0e-3 floor is about one part in twenty of headroom, which is why adoption is gated on a measured per-shape error bound rather than applied unconditionally.

5. Experimental Setup

5.1 Hardware and software

Component	Specification
GPU	NVIDIA GeForce RTX 4080 Laptop GPU — 11.99 GiB VRAM, 58 SMs, compute capability 8.9 (Ada)
CPU	Intel Core i9-14900HX — 24 cores / 32 threads
RAM	31.7 GB
Storage	SSD (HFS001TEJ9X125N)
OS	Windows 11 Home Single Language (10.0.26200)
Python	3.13.0
PyTorch	2.6.0+cu124
CUDA	12.4
Triton	triton-windows 3.2.0.post21

Table 4: Evaluation environment.

The 11.99 GiB VRAM ceiling is not incidental. It is the binding constraint on shape #14 and the reason the slicing strategy exists. Results should be read against this card.

One practical obstacle is worth recording: official Triton publishes no Windows wheel, so torch 2.6.0+cu124 on Windows ships without it. The Triton kernel was dead code until this was diagnosed and resolved with the community triton-windows build (the 3.2.x series pairs with torch 2.6).

5.2 Correctness criterion

Applied per output element by the harness, and deliberately stricter than torch.isclose, which uses $\text{atol} + \text{rtol} * |\text{ref}|$ and is more permissive:

$$\text{abs}(\text{user} - \text{ref}) \leq 0.002 \quad \text{OR} \quad \text{abs}(\text{user} - \text{ref}) \leq 0.02 * \text{abs}(\text{ref})$$

Five accuracy trials per shape, each with independently generated input.

5.3 Timing methodology

CUDA events on the current stream; 100 warmup iterations, 100 repeats per round, 3 rounds per shape, with baseline and optimized alternating between rounds to reduce thermal and clock-order bias. Calibration happens during warmup and is excluded from all reported figures.

One point proved important: the harness submits every iteration before a single synchronization. Our calibration timer had to mirror that exactly - synchronizing per call drains the launch queue and lets the host catch up, making launch-bound shapes measure artificially cheap. On shape #13 that is 6.77 ms against 10.92 ms for identical code.

6. Results

6.1 Main results

#	Shape (B, d, H, S, L, ffn)	Acc.	Speedup	Base ms	Opt ms	Path	Precision
1	64, 128, 4, 128, 4, 128	PASS	5.199x	1.863	0.358	graph x1	fp16/fp32
2	1, 128, 4, 128, 4, 128	PASS	22.522x	2.652	0.118	graph x1	fp16/fp32
3	4, 128, 4, 128, 4, 128	PASS	12.177x	2.606	0.214	graph x1	fp32
4	16, 128, 4, 128, 4, 128	PASS	8.561x	2.630	0.307	graph x1	fp32
5	128, 128, 4, 128, 4, 128	PASS	7.207x	4.605	0.639	graph x1	fp16/fp32
6	10000, 128, 4, 128, 4, 128	PASS	11.554x	676.601	58.561	eager x66	fp16/fp32
7	64, 32, 4, 128, 4, 32	PASS	11.156x	2.193	0.197	graph x1	fp16/fp32
8	64, 1024, 4, 128, 4, 1024	PASS	2.707x	21.904	8.093	eager x4	fp16/fp32
9	64, 128, 1, 128, 4, 128	PASS	5.391x	1.921	0.356	graph x1	fp16/fp32
10	64, 128, 2, 128, 4, 128	PASS	6.615x	2.242	0.339	graph x1	fp16/fp32
11	64, 128, 16, 128, 4, 128	PASS	20.933x	11.125	0.531	graph x1	fp16/fp32
12	64, 128, 4, 32, 4, 128	PASS	12.629x	2.444	0.194	graph x1	fp16/fp32
13	64, 128, 4, 1024, 4, 128	PASS	35.179x	165.528	4.705	eager x4	fp16/fp32

Table 5: The 13 shapes with a runnable reference. dtype float32, causal, padding_ratio 0.0. "graph" = CUDA-graph capture; xN = batch slice count. Shape #14 has no reference to compare against and is reported separately in Section 6.2.

13 of 13 comparable shapes PASS. Geometric-mean speedup 10.014x, minimum 2.707x (#8), maximum 35.179x (#13).

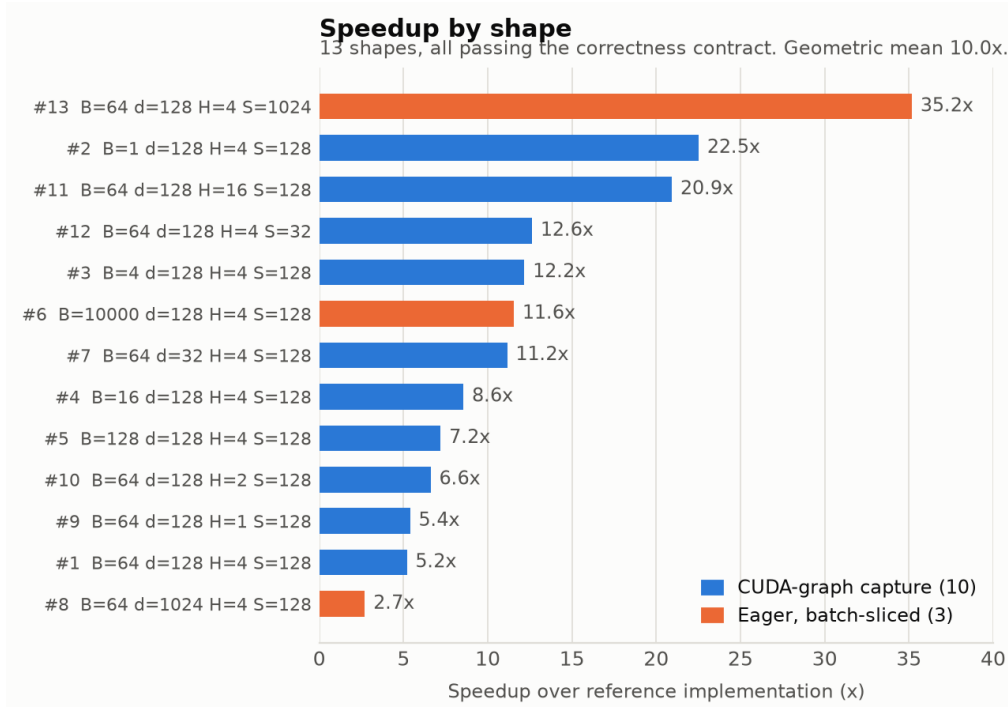


Figure 1: Speedup by shape, sorted. Colour marks the execution path chosen by runtime calibration - not a property of the shape, but a decision the implementation made after timing both options.

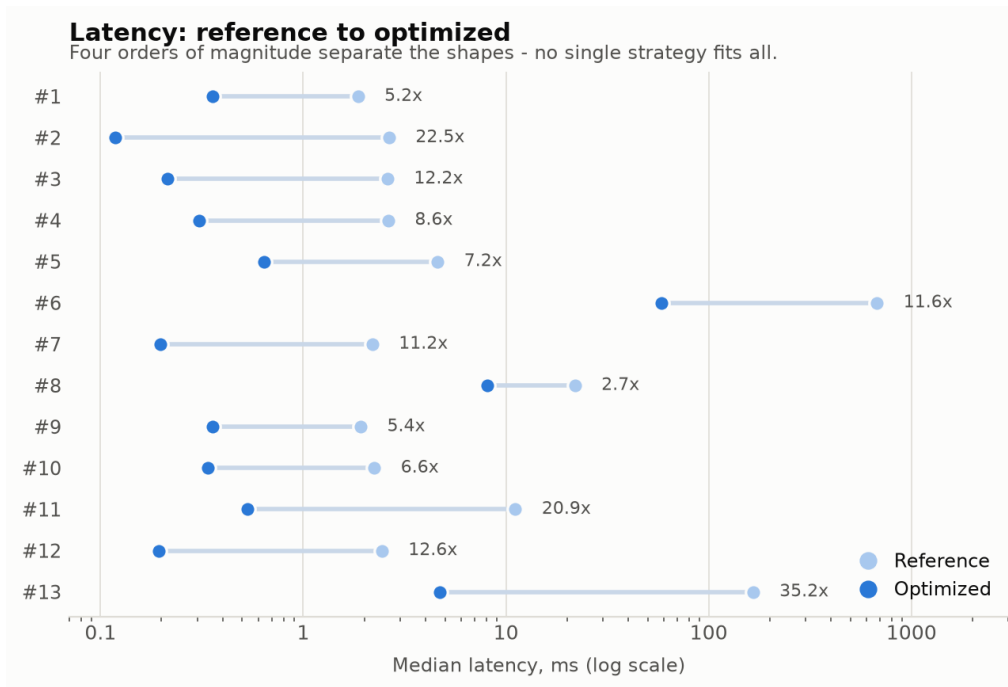


Figure 2: Reference and optimized median latency per shape. Log scale; the suite spans four orders of magnitude, which is why one fixed strategy cannot serve it.

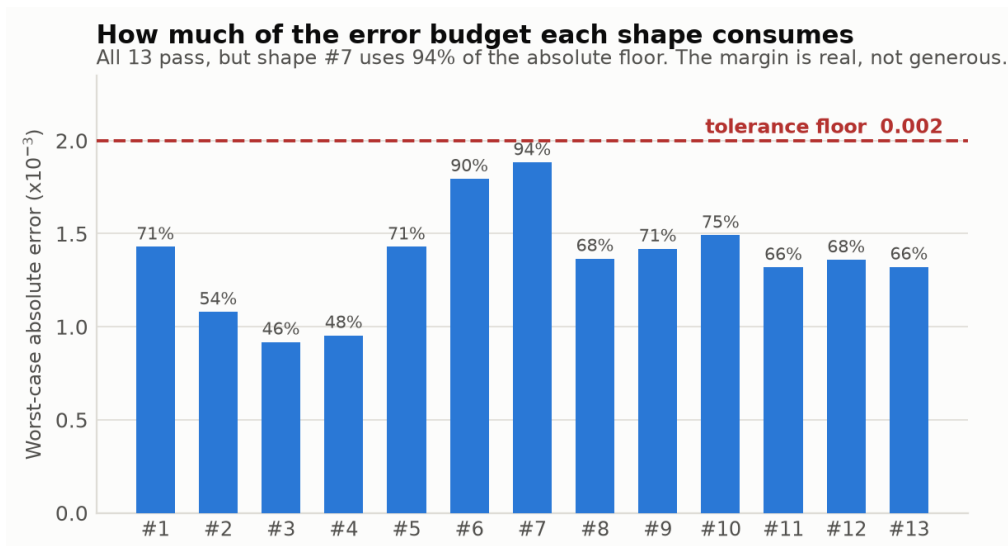


Figure 3: Worst-case absolute error per shape against the 0.002 tolerance floor, with the percentage of the budget consumed. Every shape passes, but shape #7 consumes 94% of it.

Figure 3 shows what a PASS column conceals: the mixed-precision path passes with between 46% and 94% of the absolute error budget consumed, #7 and #6 at 94% and 90%. This is the quantitative form of the caveat in Section 4.3 — sound under the stated tolerance, and it would not survive a materially tighter one.

6.2 Shape #14 — a shape with no runnable reference

Shape #14 cannot be compared. The reference materializes an explicit [B, H, S, S] score tensor; at 32 x 16 x 100,000² elements that is approximately 20 TB in fp32. No comparison and no ratio exists, so the shape is executed in isolation to demonstrate that the optimized path survives it.

```
path=eager slices=32 dtype=float16 fusion=norm+gelu
output shape (32, 100000, 1024) all finite: True
mean = 4.59837e-09 std = 0.999995
peak GPU memory allocated: 12.23 GiB
median latency 27592.766 ms over 10 repeats
```

The output statistics are the evidence that matters: mean approximately zero and standard deviation approximately one is the correct distribution for a post-LayerNorm output, so the path produces sane values rather than merely finite ones.

We deliberately publish no throughput claim for this shape. Peak allocation is 12.23 GiB against 11.99 GiB of VRAM, so part of the working set is backed by host memory and paged across PCIe. The latency is reproducible — two independent runs measured 27.59 s and 27.58 s — but what it measures is paging behaviour, not what this shape would cost resident in VRAM. The reported result is: runs to completion, output finite and correctly distributed.

Reaching even that required halving the memory floor: at fp32 the input plus a separate output buffer is 24.41 GiB. Aliasing the output onto the input needs one buffer, which is sound because batch rows are independent under attention and each slice is fully consumed before being overwritten. It is opt-in because it destroys the caller's input - unacceptable for the harness's repeated-trial loop, fine for a single-shot demonstration.

7. Ablation Study

7.1 Contribution of operator fusion

Modes are interleaved per shape rather than run as two separate sweeps: GPU clocks drift over a multi-minute run, so two sequential passes would favour whichever ran on the cooler card. 60 repeats x 2 rounds per configuration. Shapes #6 and #14 are excluded — the former for runtime, the latter for having no runnable reference.

Shape	Fusion on (ms)	Fusion off (ms)	Gain
1	0.3584	0.7311	2.04x
2	0.1208	0.2273	1.88x
3	0.2099	0.2345	1.12x
4	0.3779	0.4485	1.19x †
5	0.6431	1.1141	1.73x
7	0.1956	0.3881	1.98x
8	8.0865	9.1116	1.13x
9	0.5222	0.8284	1.59x
10	0.3369	0.7055	2.09x
11	0.5294	0.7839	1.48x
12	0.3702	0.4710	1.27x
13	4.6935	6.5684	1.40x
Total	16.444	21.612	1.31x

Table 6: Fusion ablation across 12 shapes. Accuracy PASS in all 24 runs.

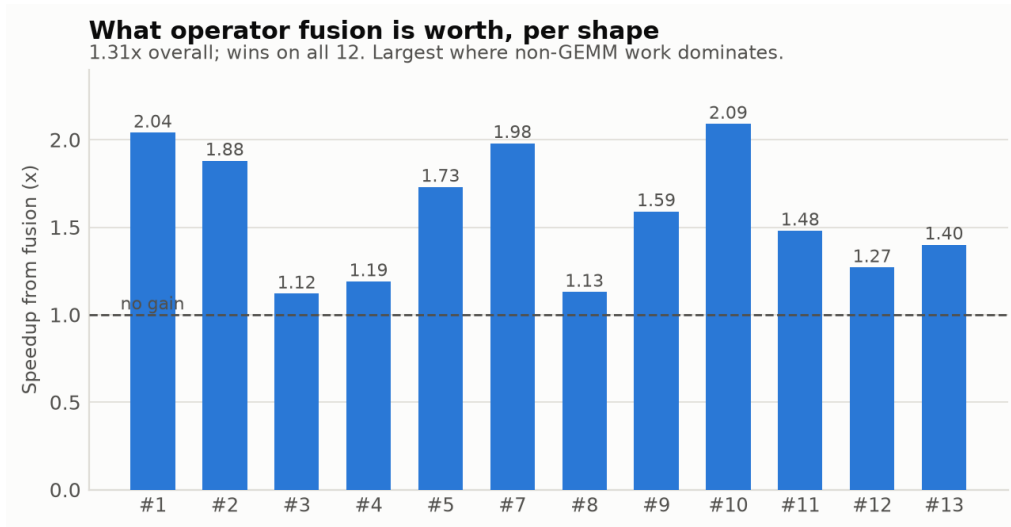


Figure 4: Per-shape speedup attributable to the two fusions. The dashed line marks parity; every shape sits above it.

The fusions are worth 1.31x and win on all 12 shapes. The gain is largest on small-to-medium shapes (#1, #2, #7, #10 near 2x), where the block's non-GEMM work is a large fraction of runtime, and smallest on #8 (1.13x), where `d_model = ffn_dim = 1024` makes the GEMMs dominate. That is the expected profile for an optimization that removes memory traffic rather than arithmetic.

† Shape #4 required re-measurement. A single sample put it at 0.78x — fusion slower — contradicting two other measurements. Five runs per mode resolved it: median 0.3779 ms on against 0.4485 ms off, with non-overlapping ranges ([0.3779, 0.4209] against [0.4475, 0.4966]). The lone 0.78x was an outlier. We record the anomaly because a one-sample reading of a borderline shape is exactly the kind of number that should not enter a results table unchallenged.

7.2 A study that overturned our own policy

The fusions were originally adopted only if they measured more than 2% faster than the eager path on that shape — a conservative rule that seemed obviously correct. A five-mode ablation showed it was losing throughput:

Policy	Total (12 shapes)	Relative
Fusion forced on	16.687 ms	1.29x
Gated: adopt only if >2% faster	17.834 ms	1.20x
Fusion off	21.468 ms	1.00x

Table 7: Five-mode study (historical; superseded by Table 5 on the current code).

Forcing beat fusion-off on all twelve shapes, so no rejection the gate made was ever correct — it discarded 41-57% of the available speedup on four shapes, and on three the "measured" choice landed below the unfused path it was meant to protect. Inverting the gate to reject only a demonstrably slower fusion changed essentially nothing (15.503 against 15.456 ms), ruling out both the threshold's magnitude and its direction.

The cause was that the measurement did not describe the path that ships. Calibration times the eager paths, but roughly half the suite then runs under graph capture, where the launch overhead the fusion removes is already gone. On shape #9 the eager comparison reports fused as 18% slower (1.09 against 0.92 ms) while the captured end-to-end result reports 34% faster (0.48 against 0.73 ms). Both are correct; they answer different questions, and the gate consulted the wrong one. Gating on a non-predictive proxy is worse than not gating, so the gate was removed rather than retuned.

8. Discussion

8.1 The measured policy is not uniform

The clearest evidence that runtime calibration earns its cost is the partition it produces across the suite:

- **Execution path.** Ten of thirteen shapes adopt CUDA-graph capture; three run eagerly. The eager three are precisely those requiring batch slicing (#6 at 66 slices, #8 and #13 at 4).
- **Precision.** Eleven of thirteen adopt mixed fp16. Shapes #3 and #4 measure fp32 as faster and use it — small-batch configurations where conversion overhead is not repaid.
- **Slice count.** Ranges from 1 (unsliced) to 66, derived from L2 capacity rather than specified.

No constant predicts this partition, and we would not have guessed it. The fp32 selection on #3 and #4, neighbours of several shapes that do adopt fp16, is exactly the boundary a hand-written threshold places wrongly.

9. AI-Assisted Development

The problem statement encourages AI-assisted development and asks for an account of the skills and tools used. This section gives that account concretely, including where AI assistance did not help and where it actively produced a wrong answer that measurement had to catch.

9.1 Tooling

Tool	Version / model	Role
Claude Code	Claude Opus 5	Primary development agent: analysis, implementation, instrumentation, debugging, documentation
VS Code	-	Editor; Claude Code ran as the integrated agent
PyTorch	2.6.0+cu124	Reference and optimized paths, SDPA, graph capture
Triton	triton-windows 3.2.0.post21	The fused residual+LayerNorm+cast kernel
cuBLASLt	via torch._addmm_activation	Fused GEMM+GELU epilogue
matplotlib / python-docx	3.x / 1.x	Figures, and this report - generated from a script so it rebuilds when a measurement changes

Table 8: Development tooling.

9.2 How AI assistance was actually used

The naive account of AI-assisted development is that the model wrote the code. That is not what happened, and it would not have produced this result. The nine optimizations are standard and documented; the hard part was deciding which apply to which shape on which hardware, and that is a measurement problem.

AI assistance was therefore most valuable in making measurement cheap. Purpose-built diagnostics -- the fusion-gate probe, the interleaved ablation harness, the shape #4 repeat loop -- took minutes rather than an afternoon, so we measured in situations where we would otherwise have reasoned and moved on. Human judgment stayed with what to measure, which hypothesis each test had to falsify, and the architectural constraint that the optimization be a mixin so weight copying keeps working.

9.3 Case studies

Four defects are worth recording in detail. All four were invisible to reading and obvious to measuring. Two of them were our own fixes being wrong on the first attempt.

9.3.1 A tolerance finer than the format could represent

The Triton kernel is verified against `F.layer_norm` before adoption and rejected if the results differ by more than 5% of the task's absolute tolerance — a budget of $1e-4$. But the comparison is performed on tensors already

stored in fp16, which quantizes in steps of approximately $9.77\text{e-}4$ near 1.0. Two results agreeing perfectly in fp32 therefore land either bit-identical or one full step apart, with nothing in between. The gate could only ever pass an exactly-equal result, and a single-ULP difference in reduction order read as a broken kernel.

The failure was silent in the worst way: the fusion reported "off" across the whole suite, indistinguishable from the kernel never running. Instrumenting the gate showed two of three cases matching at exactly $0.000\text{e+}00$ and the third at $9.766\text{e-}4$ — one fp16 ULP. The fix sizes the allowance against the storage dtype's step.

The fix was then itself tested adversarially. A loosened correctness gate that accepts everything is worse than the original bug, so we verified the repaired gate still rejects deliberately sabotaged kernels — one with a wrong epsilon, producing bad variance, and one with a 1% output scale error. Both were correctly refused.

9.3.2 A gate measuring a configuration that never shipped

Described in Section 7.2. Briefly: the fusion adoption gate timed the eager path, but half the suite runs under graph capture, so the gate was consulting a measurement that did not describe the shipping configuration. Our first fix — inverting the gate's direction — changed nothing, which is what ruled out the threshold as the cause and forced us to look at the measurement instead.

9.3.3 A regression introduced by a fix

With fusion enabled unconditionally, calibration began preferring graph capture for shape #6 — a 66-slice graph it had never captured before. Capture cost ~ 23 s of setup and its retained memory pool slowed the co-resident reference by $\sim 2\text{x}$, pushing the shape past the runner's 1800 s timeout; measured directly, capture made the optimized path slower, 95.4 ms against 65.0 ms eager. Capture is now restricted to unsliced shapes. A clean example of a fix in one place surfacing a latent defect in another.

9.3.4 A crash inside an error path

The out-of-memory preflight check's argument order did not match its format string, so a dtype landed on an integer conversion and the intended RuntimeError became a TypeError. It had never been caught because the path had never executed — the diagnostic existed for a situation that had not yet arisen. It surfaced the first time shape #14 ran through the full sweep.

9.4 Practices that proved effective

- **Measure, never assume.** Every performance claim here traces to a measurement on this hardware. Where a measurement was unavailable we say so rather than estimating — shape #14 carries no throughput number for exactly that reason.
- **Design the falsifying test first.** Each debugging step began by asking what observation would prove the current hypothesis wrong, then building it. Two hypotheses died this way.
- **Test the test.** When the correctness gate was loosened, the repaired gate was tested against deliberately broken kernels to confirm it still rejected them. A check that passes everything is not a check.
- **Control confounds, re-measure anomalies.** Ablation modes are interleaved per shape so clock drift is common to all columns; and shape #4, which measured 0.78x from one sample, was re-measured five times per mode and reversed sign.

9.5 An honest assessment

AI assistance was decisive for iteration throughput, not for the ideas. None of the nine optimizations required invention. What changed was the cost of checking, and three of the four defects in Section 9.3 were found in situations where we would otherwise have reasoned and moved on.

It is worth being explicit that AI assistance also produced wrong answers. The inverted fusion gate was a confidently-argued fix that measurement showed changed nothing. The initial shape #4 ablation figure was published from a single sample and had the sign wrong. In both cases the error was caught by the measurement discipline rather than by review, which is the argument for the discipline: the assistant is a fast generator of plausible hypotheses, and plausibility is exactly what a measurement is for.

10. Limitations and Future Work

- **Shape #14 is not a throughput result.** Peak allocation exceeds VRAM, so part of the working set pages over PCIe. The latency is reproducible but reflects paging, not resident performance. Fitting it needs a smaller slice budget or more memory.
- **Calibration measures the wrong configuration.** Calibration times the eager path while roughly half the suite runs under graph capture. We removed the fusion speed gate rather than making the measurement predictive; calibrating under capture is the correct fix and the first thing we would pursue with more time.
- **Single-GPU validation.** Decisions are made at runtime rather than baked in, so the approach should transfer, but we have verified it only on an RTX 4080 Laptop. The remaining constants are grounded in measurements from this one card.
- **The fp16 margin is thin.** Roughly one part in twenty of headroom against the absolute floor. A tighter tolerance would force several shapes back to fp32 and forfeit most of their speedup.
- **Removed as unmeasured.** A multi-stream slice path was implemented and then removed because it was never benchmarked. An untested concurrency path is worse than none.
- **No hand-written attention kernel.** We rely on PyTorch's SDPA, which reaches FlashAttention. Correct under time constraints, but a kernel specialized to these shapes is where the remaining headroom most plausibly lies.

11. Conclusion

We achieve a 10.014x geometric-mean speedup across the thirteen comparable shapes of the TechJam 2026 Problem 3 suite, correct on every one under the task's tolerance, and additionally execute the fourteenth shape — for which no reference implementation can physically run — to a correct and well-distributed output.

The more transferable result is methodological. Optimizing across a heterogeneous shape suite invites hardcoded shape checks, and the problem statement permits them. Deferring those decisions to runtime measurement produced a policy we would not have written by hand — fp32 on two shapes surrounded by fp16 neighbours, capture on ten of thirteen, slice counts from 1 to 66 — and exposed our own assumptions as wrong more than once. The fusion gate is the sharpest example: a rule that looked obviously correct was discarding up to 57% of the available speedup on individual shapes, because it timed a configuration that never shipped.

References

- [1] Vaswani, A. et al. Attention Is All You Need. NeurIPS, 2017.
- [2] Dao, T. et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS, 2022.
- [3] Dao, T. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. ICLR, 2024.
- [4] Tillet, P., Kung, H. T., Cox, D. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. MAPL, 2019.
- [5] PyTorch Documentation. `torch.nn.functional.scaled_dot_product_attention`.
- [6] NVIDIA. CUDA C++ Programming Guide — CUDA Graphs.
- [7] NVIDIA. cuBLAS Library User Guide — cuBLASLt Epilogues.
- [8] Micikevicius, P. et al. Mixed Precision Training. ICLR, 2018.
- [9] Ba, J. L., Kiros, J. R., Hinton, G. E. Layer Normalization. arXiv:1607.06450, 2016.

Appendix A: Ablation Toggles

Every optimization is individually switchable at runtime, so any claim in this report can be checked by disabling the thing it credits: `TJ_DISABLE_FUSION`, `TJ_DISABLE_FUSED_NORM`, `TJ_DISABLE_GELU_EPILOGUE`, `TJ_DISABLE_SDPA`, `TJ_DISABLE_GRAPH`, `TJ_DISABLE_FUSED_QKV`, `TJ_DISABLE_ELISION`, `TJ_PRECISION`, `TJ_CHUNK_MB`, `TJ_INPLACE_OUTPUT`. The full list with descriptions is at the top of `optimized_transformer.py`.

Appendix B: Reproduction

```
pip install torch --index-url https://download.pytorch.org/whl/cu124
pip install triton-windows==3.2.0.post21 # Windows; 'triton' on Linux

python run_all_shapes.py # full sweep, ~15 min
python run_optimized_only.py --inplace-output # shape #14
```

The sweep takes roughly fifteen minutes, the majority of it in shape #6, where the reference implementation is the slow component. Expect 13 of 14 shapes to pass, with #14 reported separately as described in Section 6.2.

Source, full commit history, and the in-repository technical report:
github.com/koompaj/Tiktok_Techjam2026_Blackroom